Contrastive Reinforcement Learning

Linus Rattay, Julian Menon
https://github.com/JulianPMenon/Luck-base-reinforcement-learning

Abstract

We propose a contrastive CNN structure to solve different sparse reward MiniGrid problems. To do so we use a contrastive encoder, a DQN and a memory bank for intrinsic reward. Our approach seems to be able to outperform RNDs in regards to average rewards.

1 Introduction

In this paper we aimed to use a contrastive CNN with a momentum encoder similar to CURL Srinivas et al. [2020] or MoCo He et al. [2020] to improve Reinforcement Learning in sparse reward environments over a baseline. We used a Deep Q-Learning Network (DQN) for off-policy learning in the sparse reward environment MiniGrid Chevalier-Boisvert et al. [2023]. The DQN was trained with the encoding we got by pretraining our two encoder CNN with InfoNCE as the contrastive loss function. We then chose a RND as a baseline to perform the same task for comparison. Our approach was better in learning on the chosen MiniGrid levels than our RND baseline.

2 Related Work

2.1 CURL - Contrastive Unsupervised representations for Reinforcement Learning

CURL is using contrastive learning for state representations for RL. CURL Architecture (Figure 1): "A batch of transitions is sampled from a replay buffer. Observations are then data-augmented twice to form query and key observations, which are then encoded with the query encoder and key encoders, respectively. The queries are passed to the RL algorithm while query-key pairs are passed to the contrastive learning objective. During the gradient update step, only the query encoder is updated. The key encoder weights are the moving average (EMA) of the query weights." Srinivas et al. [2020]

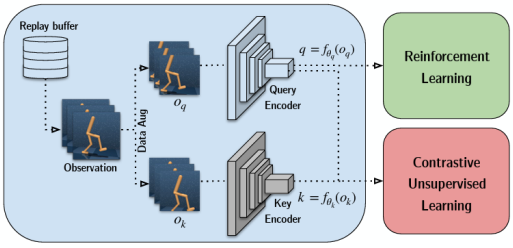

Figure 1: Architecture of CURL using two encoders. On static and one updated every gradient update step to calculate loss. simultaneously reinforcement learning and Contrastive learning. Srinivas et al. [2020]

3 Approach

Our approach consists of a contrastive encoder, a DQN and a Memory Bank used to calculate an intrinsic reward.

3.1 Contrastive Learning

The key idea of our approach is to learn a contrastive encoder to represent the search space for a DQN. We collected training data by running a set amount of games and choosing random actions in each game state. Each game state is a data point in the training set. Afterwards we used that data to train a Contrastive CNN with two encoders and projection heads inspired by CURL, while the key encoder is used as a momentum encoder. As a loss function we used InfoNCE.

3.2 DQN

Our rl-agent is structured like a DQN, with a target and a q-network. We used a straight forward DQN consisting of only Linear and ReLU layers, epsilon Greedy policy and L1 loss.

3.3 Memory bank and intrinsic reward

Because we could not use the two encoder setup while reinforcement learning like in CURL Srinivas et al. [2020], because of resource constrains, we had to pre train our encoder setup and store intrinsic rewards as memories in a memory bank. For this we stored the encodings of the environment observations as a list using the trained query encoder of our contrastive CNN. The intrinsic reward of a state is the state entropy calculated using k-nearest neighbors on the the observation during training of the DQN.

3.4 Hindsight Experience Replay - HER

To mitigate problems with sparse reward problem we used Hindsight Experience Replay Andrychowicz et al. [2018]. It helps by assigning intermediate rewards in our search space by sampling from the state transitions.

3.5 Structure of one Contrastive RL run for an environment

Algorithm 1 Contrastive RL with Intrinsic Reward and HER

Require: environment e, agent A
  Collect data for contrastive learning from e
  Train contrastive encoder on collected data
  Build memory bank $\mathcal{M}$ from encoded observations
  for each episode **do**
    Initialize $s_0 \sim e$
    for each step **do**
      Encode state: $z_t \leftarrow \text{encoder}(s_t)$
      Select action $a_t \leftarrow A(z_t)$
      Execute a_t in e, observe s_{t+1}, r_t
      Compute intrinsic reward r_t^{int} using $\mathcal{M}$
      Store transition $(z_t, a_t, r_t, z_{t+1}, r_t^{\text{int}})$
      Update agent via experience replay
    end for
    Apply Hindsight Experience Replay (HER) to episode transitions
    Update target network of A
    Decay exploration parameter of A
  end for
  return policy π

4 Experiments

We ran experiments on two difficulties and compared our approach with HER, without HER and the RND-Baseline using student's T-test.

Repository: https://github.com/JulianPMenon/Luck-base-reinforcement-learning | Menon and Rattay [2025]

4.1 Sparse Rewards Environment - MiniGrid

We used MiniGrid Chevalier-Boisvert et al. [2023] because we thought it is easy to implement for us easy to understand. We chose different levels where we thought that they have different difficulties for our agent. We chose Empty8v8 since it is the most basic one. We then first chose variants for door key but realized that our agent had difficulties ever reaching the goal. Therefore we chose LavaGap as a harder Level.

4.2 Hyperparameter Tuning with Randomsearch based Hyperband

We build a random search based hyperband algorithm inspired by the one we used in the AutoML project. We used it to find good Hyperparameter configurations for both the DQN and the RND. For the RND we searched for learning rate, feature dimension and batch size. For the DQN we searched for epsilon, epsilon decay and gamma. There we set the learning rate to 0.00001. We searched for the DQN with and without the HER implementation.
Configurations found:

- RND on easy environment:
  - Learning rate: 0.000044
  - Feature dimension: 192
- RND on hard environment:
  - Learning rate: 0.000154
  - Feature dimension: 128
- DQN on easy environment:
  - Epsilon: 0.6965049505233765
  - Epsilon Decay: 0.908572554588317
  - Gamma: 0.9351036548614502
- DQN on hard environment:
  - Epsilon: 0.6965049505233765
  - Epsilon Decay: 0.908572554588317
  - Gamma: 0.9351036548614502
- DQN on easy environment without HER:
  - Epsilon: 0.9151939749717712
  - Epsilon Decay: 0.9602900147438049
  - Gamma: 0.8741558790206909

4.3 Experiment: Results

We ran RND and our approach with and without HER on the Minigrid environment MiniGrid-Empty-8x8-v0 using the seeds from 0 to 9 as an Easy task. To further test our approach we tested it the harder task MiniGrid-LavaGapS7-v0 with the goal to find a path threw the lava. Both results are shown in Table 1. Our approach seems to be able to solve both tasks quite well. But HER as a big impact on our rewards. Without HER our approach runs less consistent in finding good solutions.

Environment	RND	DQNnoHER	DQNwithHER
MiniGrid-Empty-8x8-v0	0.335 ± 0.021	0.386 ± 0.135	0.842 ± 0.051
MiniGrid-LavaGapS7-v0	0.031 ± 0.006	-	0.141 ± 0.026

Table 1: Shows the mean avg reward of the RND and our approach with and without HER over the seed from 0 to 9 on the easy task Empty and harder task LavaGap.

4.4 T-test

We calculated a one-tailed Welch's t-test ($\alpha = 0.05$) for both the easy and hard tasks using the best average reward values from ten seeds run for the RND baseline and our approach. Additionally, we calculated it for the easy task comparing our approach with and without the HER implementation. Our hypothesis was that, in both the hard and easy tasks, our approach is at most as good as the RND at learning on MiniGrid. We could reject this hypothesis: our approach outperformed the RND baseline in both the easy ($t(17.99) = 13.82$, $p < 0.0001$) and hard environments ($t(11.34) = 8.93$, $p < 0.0001$). We see this reflected in the fact that all best average rewards of our approach are higher than those of the RND. However, we could not reject the hypothesis that our approach without the HER implementation gets at most the same reward ranges as the RND ($t(14.37) = 0.81$, $p = 0.2165 > \alpha = 0.05$). Our approach with HER significantly outperformed the version without HER ($t(13.14) = 6.99$, $p < 0.0001$) regarding average rewards. This suggests that the reward improvement of our Contrastive RL approach over the RND baseline may be due to the HER implementation rather than the contrastive component alone.

4.5 Hardware

We performed the experiment on two separate computers with CPU only, due to nonexistent or unsupported GPUs. One had an 8th Gen Intel i7 vPro and 16GB RAM, and the other an AMD Ryzen 7 2700X 8-core with 16GB RAM. The Hyperband was additionally run on Google Colab with a T4 GPU to save time.

4.6 Limitations

We could only run a few training experiments with our approach because of hardware and time constraints. One full experiment run with 10 seeds for a single level, each with 400 episodes, took approximately 4 hours on the weaker CPU. The approach could not be tested with high episode counts or many alterations to confirm stable results. The experiment was only performed on EmptyGrid and LavaGap and does not generalize to other levels. Especially DoorKey, which we also tried, was excluded very early because it appeared not to work with our setup. The improvements over RND could be due to hyperparameter tuning and the inclusion of HER. With more time for hyperparameter optimization, we might find configurations for both RND and our approach that change the results, considering that only some, but not all, hyperparameters were tuned using a random search algorithm. The rest were kept constant.

5 Discussion

We created a contrastive DQN model based on CURL and HER that was able to outperform RND with 99% confidence. However, we were not able to show that our approach outperforms RND without HER. Future research might investigate whether our approach performs differently if the encoder is trained simultaneously with the DQN. Another direction could be studying the behavior of our approach in environments with denser rewards. Since using HER significantly improved our approach, comparing against other methods using HER could be insightful.

6 Ai usage

We used ChatGPT 5 and 4.1 for spellchecking parts of this paper. GitHub Copilot, together with ChatGPT 4.1, was used to debug and comment on parts of the code, such as HER and t-tests, as well

as to fix bugs in the training loop of our experiment. Additionally, ChatGPT was used to find papers on ResearchGate more quickly.

7 Acknowledgment

Thank you to the AutoML Group lead by Prof. Dr. rer. nat. Marius Lindauer for providing the base and learning environment for this project with the two Lectures Reinforcement Learning and Automated Machine Learning at Leibniz University Hannover.

References

Marcin Andrychowicz, Filip Wolski, Alex Ray, Jonas Schneider, Rachel Fong, Peter Welinder, Bob McGrew, Josh Tobin, Pieter Abbeel, and Wojciech Zaremba. Hindsight experience replay, 2018. URL https://arxiv.org/abs/1707.01495.

Maxime Chevalier-Boisvert, Bolun Dai, Mark Towers, Rodrigo Perez-Vicente, Lucas Willems, Salem Lahlou, Suman Pal, Pablo Samuel Castro, and Jordan Terry. Minigrid & miniworld: Modular & customizable reinforcement learning environments for goal-oriented tasks. In *Advances in Neural Information Processing Systems 36, New Orleans, LA, USA*, December 2023.

Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. Momentum contrast for unsupervised visual representation learning, 2020. URL https://arxiv.org/abs/1911.05722.

Julian Menon and Linus Rattay. Luck base reinforcement learning - github, 2025. URL https://github.com/JulianPMenon/Luck-base-reinforcement-learning.

Aravind Srinivas, Michael Laskin, and Pieter Abbeel. Curl: Contrastive unsupervised representations for reinforcement learning, 2020. URL https://arxiv.org/abs/2004.04136.

Checklist

This checklist is not mandatory, but can help you set up your project in a reproducible manner. Options for filling it out are: [Yes] [No] [NA]

1. General points:
   (a) Do the main claims made in the abstract and introduction accurately reflect your contributions and scope? [Yes]
   (b) Did you cite all relevant related work? [Yes]
   (c) Did you describe the limitations of your work? [Yes]
   (d) Did you include a discussion of future work? [Yes]
2. If you are including theoretical results...
   (a) Did you state the full set of assumptions of all theoretical results? [Yes]
   (b) Did you include complete proofs of all theoretical results? [Yes]
3. If you ran experiments (e.g. for benchmarks)...
   (a) Did you include the code, data, and instructions needed to reproduce the main experimental results (either in the supplemental material or as a URL)? [Yes]
   (b) Did you specify all the training details (e.g., data splits, hyperparameters, how they were chosen)? [Yes]
   (c) Did you run at least 10 repetitions of your method? [Yes]
   (d) Did you report error bars (e.g., with respect to the random seed after running experiments multiple times)? [Yes]
   (e) Did you include the total amount of compute and the type of resources used (e.g., type of GPUs, internal cluster, or cloud provider)? [Yes]
4. If you are using existing assets (e.g., code, data, models) or curating/releasing new assets...
   (a) If your work uses existing assets, did you cite the creators? [Yes]
   (b) Did you make sure the license of the assets permits usage? [Yes]
   (c) Did you reference the assets directly within your code and repository? [NA]